

MSPM0G3507

Table 1-1. Key Features

Feature	MSPM0Gx Series
Core / Frequency	CM0+ / 80 MHz
Supply Voltage	1.62 V to 3.6 V
Temperature	-40°C to 125°C
Memory	128KB to 32KB
RAM	Up to 32KB
GPIO (max)	60
Analog	2x 4-Msps 12-bit ADC
	3x high-speed comparators
	2x op amps
	1x 12-bit DAC

Table 1-1. Key Features (continued)

Feature	MSPM0Gx Series
Communication (max)	2x SPI
	2x I2C Fast+
	4x UART (LIN)
	1x CAN-FD
Timers	7
Advance Timers[HJ2]	Yes (2x)
Hardware Accelerator	Optional
Security	CRC, TRNG, AES256
Low power	Active: 85 µA/MHz
	Standby (RTC): 1.5 µA



Setup



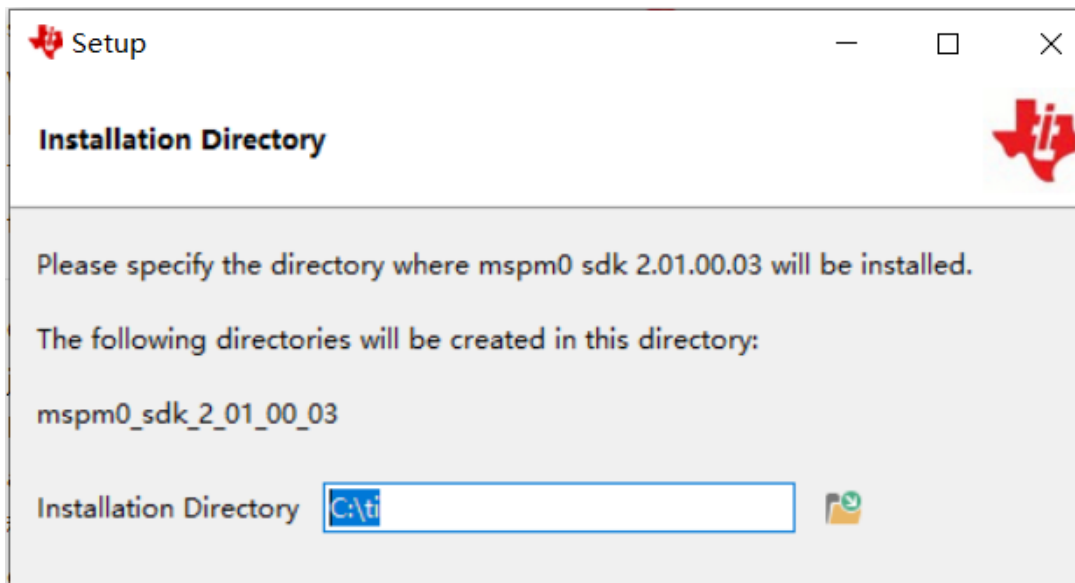
Installation Directory



Please specify the directory where CCS Theia 1.4.1.00001 will be installed.

Installation Directory





输入引脚上拉 把不定态默认为高电平

```
int main(void)
{
    SYSCFG_DL_init();

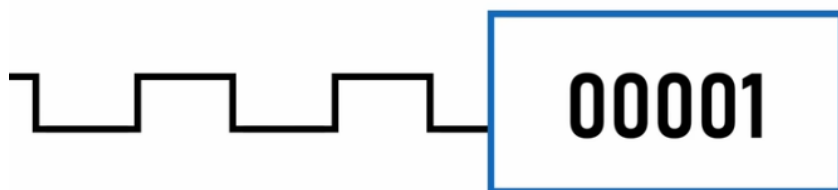
    while (1) {
        if (DL_GPIO_readPins(GPIO_BUTTON_PORT, GPIO_BUTTON_PIN_BUTTON_S2_PIN)){
            DL_GPIO_setPins(GPIO_LED_PORT, GPIO_LED_PIN_LED_1_PIN);
        }
        else{
            DL_GPIO_clearPins(GPIO_LED_PORT, GPIO_LED_PIN_LED_1_PIN);
        }
    }
}
```

DL_GPIO_readPins 检测外部输入

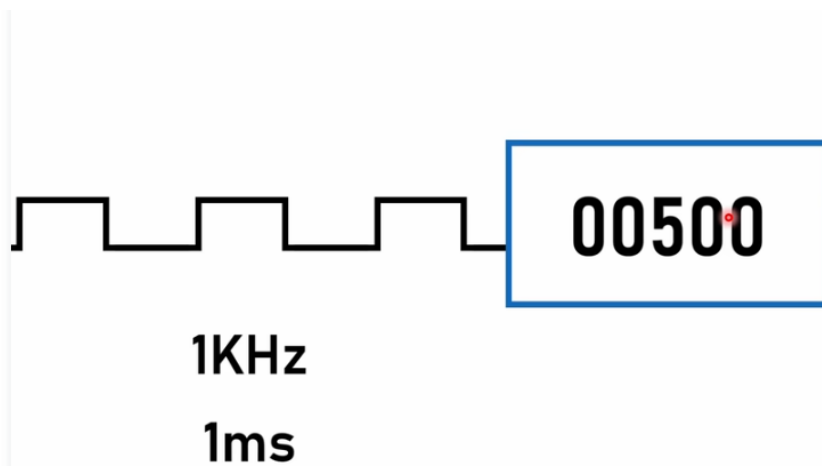
GPIO翻转

```
while (1) {
    /*
     * Call togglePins API to flip the current value of LEDs 1-3. This
     * API causes the corresponding HW bits to be flipped by the GPIO HW
     * without need for additional R-M-W cycles by the processor.
     */
    delay_cycles(DELAY);
    DL_GPIO_togglePins(GPIO_LEDS_PORT,
        GPIO_LEDS_USER_LED_1_PIN | GPIO_LEDS_USER_LED_2_PIN |
        GPIO_LEDS_USER_LED_3_PIN | GPIO_LEDS_USER_TEST_PIN);
}
```

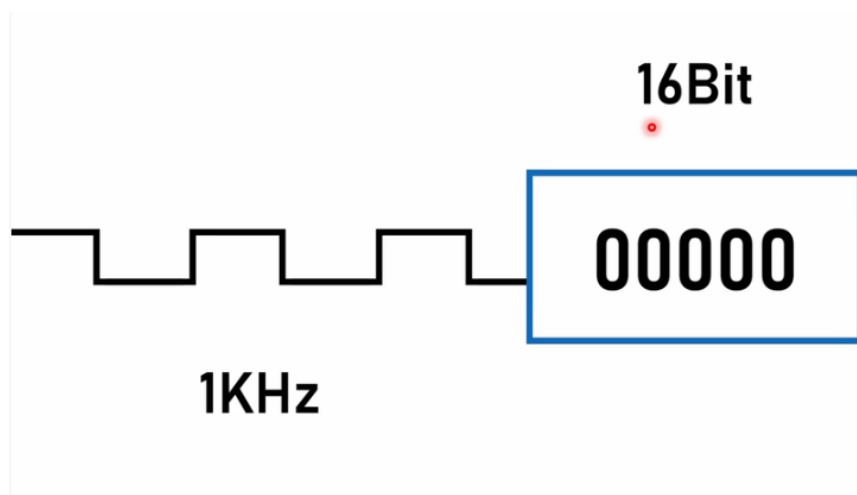
定时器本身就是计数器



加法计数器



减法计数器



最短计时时间

1ms

最长计时时间

65535ms

时钟周期 和 计数器位数决定

- **Intelligent digital peripherals**
 - 7-channel DMA controller
 - Math accelerator supports DIV, SQRT, MAC and TRIG computations
 - Seven timers supports up to 22 PWM channels
 - One 16-bit general-purpose timer

- One 16-bit general-purpose timer supports QEI
- Two 16-bit general-purpose timers support low-power operation in STANDBY mode
- One 32-bit high-resolution general-purpose timer
- Two 16-bit advanced timers with deadband support up to 12 PWM channels

五个基础定时器 两个高级定时器

Time Profiles 中设置 Periodic with 500 ms Period and Zero Event 什么意思



在MSPM0G3507单片机的定时器配置中，**Time Profiles** 中的 **Periodic with 500 ms Period and Zero Event** 选项通常是指定时器的计时方式和触发事件的配置。具体来说：

1. Periodic with 500 ms Period:

- **Periodic** 意味着定时器会以固定的周期运行，也就是说它会在每个周期结束时自动重新加载并开始下一个周期。
- **500 ms Period** 指定了定时器的周期时间为500毫秒。这意味着定时器会每隔500毫秒触发一次事件，比如中断。

2. Zero Event:

- **Zero Event** 指的是定时器在计数到达零（或者预设值）的时刻产生的事件。在这个配置中，定时器会在每个500毫秒的周期结束时，计数器归零，然后触发一个**Zero Event**事件，这个事件通常会触发一个中断（比如你代码中的`DL\_TIMER\_IIDX\_ZERO`中断）。
- 这个事件可以用来执行特定的操作，比如切换LED状态，发出信号，或者启动另一个任务。

总结

当你在 **Time Profiles** 中设置 **Periodic with 500 ms Period and Zero Event** 时，这表示定时器每隔500毫秒产生一个中断，触发“零事件”（Zero Event），你可以在中断服务函数中对这一事件作出响应。

用BUSCLK

最终时钟频率

One-shot 一次

Periodic 循环

Zero Event 数到0的时候进入中断

Desired Timer Period 是你在应用中希望定时器运行的周期，而它需要在定时器的 **Timer Period Range** 内，并且受 **Resolution** 的限制。确保 **Desired Timer Period** 是分辨率的整数倍且在可支持的周期范围内，才能正确配置定时器以满足你的需求。

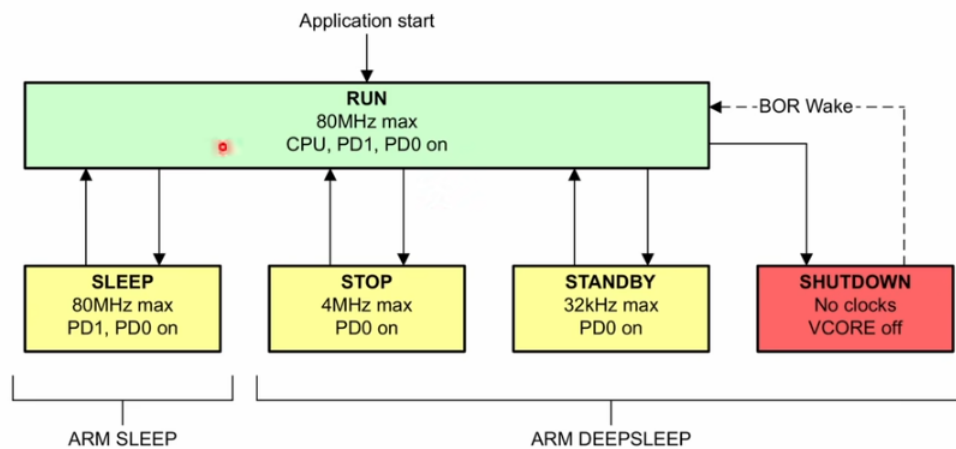


Figure 2-2. MSPM0Gxx Operating Modes

五种运行模式

```

int main(void)
{
    SYSCFG_DL_init();

    NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);    //使能定时器中断
    DL_SYSCFG_enableSleepOnExit();
    // Enables sleep on exit when the CPU moves from handler mode to thread mode.
    //系统在退出中断服务程序（ISR）时进入睡眠模式。这样可以在低功耗模式下运行系统，并在有中断时唤醒
    DL_TimerG_startCounter(TIMER_0_INST);    //启用计时器（在sysconfig里面没有start）

    while (1) {
        __WFI(); //__WFI()指令是ARM架构中的一个低功耗指令，表示“等待中断”。这会让CPU进入低功耗模式
    }
}

void TIMER_0_INST_IRQHandler(void)    //中断服务函数
{
    switch (DL_TimerG_getPendingInterrupt(TIMER_0_INST)) {    //判断哪种中断进入中断服务函数
        case DL_TIMER_IIDX_ZERO:    //如果是Zero 0 事件
            DL_GPIO_togglePins(GPIO_LEDS_PORT,
                                GPIO_LEDS_USER_LED_1_PIN | GPIO_LEDS_USER_TEST_PIN);
            break;
        default:
            break;
    }
}

```

PWM

Calculated Timer Clock Values	
Calculated Clock Frequency (Hz)	125000
Timer Clock Information	1.91 Hz to 62.50 kHz w/ resoluti

输出频率范围

PWM Mode	Edge-aligned Down Counting
PWM Channel(s)	Edge-aligned Down Counting
Enable Channel(s)	Edge-aligned Up Counting
	Center-aligned

边缘对齐

中心对齐

一个PWM多路（最多四路）信号频率一样 占空比可调

调整输出引脚

PinMux Peripheral and Pin Configuration	
Timer Peripheral	TIMG0
Capture/Compare Pin 0	PA12/5
Capture/Compare Pin 1	PA13/6

```
int main(void)
{
    SYSCFG_DL_init();

    DL_TimerG_startCounter(PWM_0_INST);

    while (1) {
        __WFI();
    }
}
```

主函数简单

可以去调用sysconfig.c函数

```
while (1)
{
    if(DL_GPIO_readPins(GPIO_BUTTON_PORT, GPIO_BUTTON_PIN_S2_PIN))
    {
        DL_TimerG_setCaptureCompareValue(PWM_0_INST, 22400, DL_TIMER_CC_0_INDEX);
        DL_TimerG_setCaptureCompareValue(PWM_0_INST, 9599, DL_TIMER_CC_1_INDEX);
    }
    else
    {
        DL_TimerG_setCaptureCompareValue(PWM_0_INST, 9599, DL_TIMER_CC_0_INDEX);
        DL_TimerG_setCaptureCompareValue(PWM_0_INST, 22400, DL_TIMER_CC_1_INDEX);
    }
}
```

ADC

VDD 3.3V

Conversion Mode	Single
Conversion Starting Address	Single
Fast Mode	Sequence

Sequence ADC多通道测量

Interrupt Configuration	
Enable Interrupts	MEM0 result loaded interrupt
Interrupt Priority	Default

把结果存放在MEMORY0中

Power Down Mode	Manual
Desired Sample Time 0	125us
Actual Sample Time 0	125.00 μs
Desired Sample Time 1	0 ms
Actual Sample Time 1	0.00 s

ADC 测量值： 0-4095

```

volatile bool gCheckADC;           //volatile禁用编译器优化
volatile uint16_t gAdcResult;

int main(void)
{
    SYSCFG_DL_init();

    NVIC_EnableIRQ(ADC12_0_INST_INT_IRQN);
    gCheckADC = false;

    while (1) {
        DL_ADC12_startConversion(ADC12_0_INST);    //ADC开始转换

        while (false == gCheckADC) {
            __WFE();
        }

        gAdcResult = DL_ADC12_getMemResult(ADC12_0_INST, DL_ADC12_MEM_IDX_0);

        if (gAdcResult > 0x7ff) {
            DL_GPIO_clearPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_1_PIN);
        } else {
            DL_GPIO_setPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_1_PIN);
        }
        gCheckADC = false;
        DL_ADC12_enableConversions(ADC12_0_INST);
    }
}

void ADC12_0_INST_IRQHandler(void)
{
    switch (DL_ADC12_getPendingInterrupt(ADC12_0_INST)) {
        case DL_ADC12_IIDX_MEM0_RESULT_LOADED: //转换完成后触发中断
            gCheckADC = true;
            break;
        default:
            break;
    }
}

```

`__BKPT(0);` :

- 这行代码通常用于触发一个断点。
- 当程序运行到这一行时，会暂停执行，并跳转到调试器的断点处。
- 这样，开发者就可以在调试器中检查程序的状态，找出问题所在。

利用程序 打了一个断点

ADC 多通道转换

Interrupt Configuration

Enable Interrupts

MEM3 result loaded interr

Interrupt Priority

Default

Sampling Mode Configuration

Conversion Mode

Sequence

Conversion Starting Address

0

Conversion End Address (Sequenc...

3

Enable Repeat Mode

☐

Sampling Mode

Auto

Trigger Source

Software

Conversion Data Format

Binary unsigned, right aligned

Conversion End Address (Sequenc...

☒ Select All
☒ ADC Conversion Memory 0
☒ ADC Conversion Memory 1
☒ ADC Conversion Memory 2
☒ ADC Conversion Memory 3
☐ ADC Conversion Memory 4
☐ ADC Conversion Memory 5
☐ ADC Conversion Memory 6
☐ ADC Conversion Memory 7
☐ ADC Conversion Memory 8

ADC Conversion Memory Configurati...

Active Memory Control Blocks

ADC Conversion Memory (

启动转换一次即可

DAC

```

uint32_t DAC_value;

SYSCFG_DL_init();

/* Set output voltage:
 * DAC value (12-bits) = DesiredOutputVoltage x 4095
 * -----
 * ReferenceVoltage
 */
DAC_value = (DAC12_OUTPUT_VOLTAGE_mV * 4095) / DAC12_REF_VOLTAGE_mV;
//最后函数里面写的是DAV_Value
DL_DAC12_output12(DAC0, DAC_value);
DL_DAC12_enable(DAC0);

```

Basic Operation Configuration

Enable DAC	☒
Positive Voltage Reference	Positive reference is VREF+ ▼
Negative Voltage Reference	Negative reference is VREF- ▼
Enable FIFO	☐

注意这里Enable勾选上了

注意要使能MFPCLK才能使能DAC12

VREF ?

+ ADD

REMOVE ALL

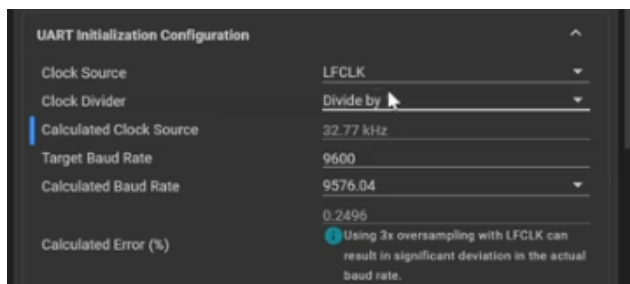
Quick Profiles

VREF Profiles	Custom ▼
---------------	----------

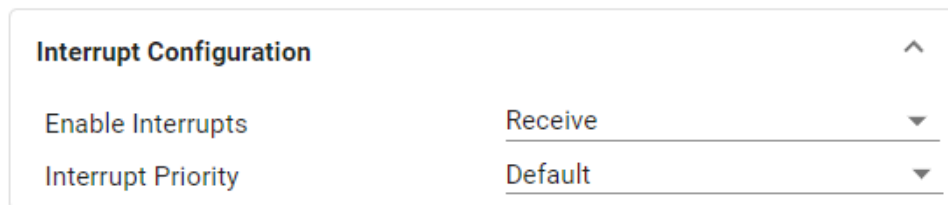
Basic Configuration

Mode	Internal ▼
VREF+/VREF- Configuration	VREF+ enabled, VREF- enabled ▼
Internal Voltage (V)	2.5 ▼
Enable Delay for VREF Startup	☒

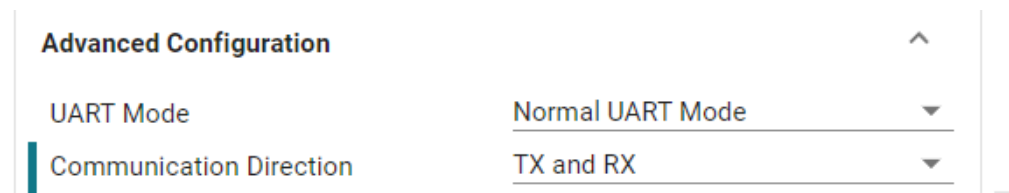
UART



波特率不高 用低频时钟即可



使能一个接受中断!



配置串口发送和接受双向

```
volatile uint8_t gEchoData = 0; //保存接受到的数据
int main(void)
{
    SYSCFG_DL_init();

    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN); //清除状态防止一上电直接进入中断
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
    DL_SYSCTL_enableSleepOnExit();

    while (1) {
        __WFI();
    }
}
```

```
void UART_0_INST_IRQHandler(void)
{
    switch (DL_UART_Main_getPendingInterrupt(UART_0_INST)) {
        case DL_UART_MAIN_IIDX_RX:
            DL_GPIO_togglePins(GPIO_LEDS_PORT,
                               GPIO_LEDS_USER_LED_1_PIN | GPIO_LEDS_USER_TEST_PIN);
            gEchoData = DL_UART_Main_receiveData(UART_0_INST); //接受数据
            DL_UART_Main_transmitData(UART_0_INST, gEchoData); //发送相同的数据
            break;
        default:
            break;
    }
}
```

PinMux Peripheral and Pin Configuration		
UART Peripheral	UART0	▼ 🔒
RX Pin	PA11/57	▼ 🔒
TX Pin	PA10/56	▼ 🔒

引脚不用改变！

```
while (1) {
}

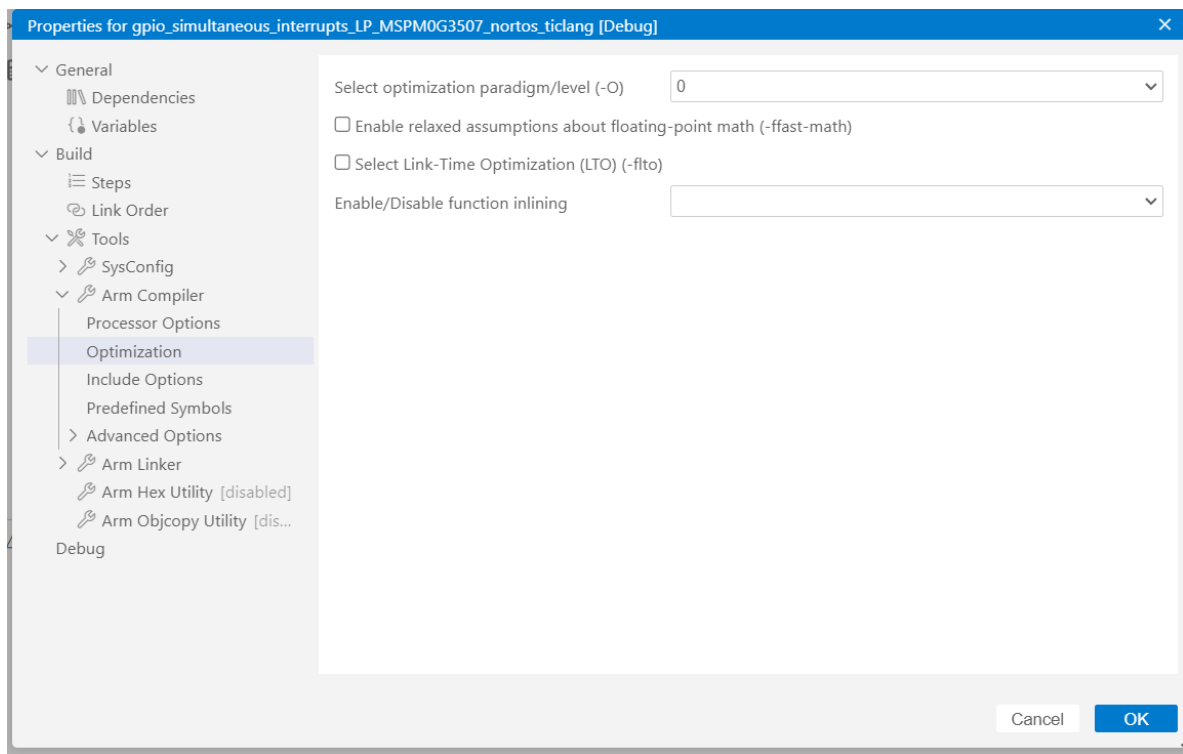
void UART_0_INST_IRQHandler(void)
{
    gEchoData[UART_Count++] = DL_UART_Main_receiveData(UART_0_INST);
    DL_UART_Main_transmitData(UART_0_INST, gEchoData);
}
```

发送和接受字符串

外部中断

Interrupts/Events		
Enable Interrupts	☒	
Interrupt Priority	Default	▼
Trigger Polarity	Trigger on Both Edges	▼
Event Publishing Channel	Disabled (0)	▼

这里双边沿触发



取消编译优化

```
int main(void)
{
    SYSCFG_DL_init();

    /* Enable Interrupt for both GPIOA and GPIOB ports */
    //对中断进行使能
    NVIC_EnableIRQ(GPIO_SWITCHES_GPIOA_INT_IRQN);
    NVIC_EnableIRQ(GPIO_SWITCHES_GPIOB_INT_IRQN);

    while (1) {
        __WFI();
    }
}
```

```
//GPIO 中断都在Group1中
void GROUP1_IRQHandler(void)
{
    /*
     * Get the pending interrupt for the GPIOA port and store for
     * comparisons later
     */
    uint32_t gpioA = DL_GPIO_getEnabledInterruptStatus(GPIOA,
        GPIO_SWITCHES_USER_SWITCH_1_PIN | GPIO_SWITCHES_USER_SWITCH_3_PIN);

    /*
     * Bitwise AND the pending interrupt with the pin you want to check,
     * then check if it is equal to the pins. Clear the interrupt status.
     */
    if ((gpioA & GPIO_SWITCHES_USER_SWITCH_1_PIN) == GPIO_SWITCHES_USER_SWITCH_1_PIN)
    {
        DL_GPIO_togglePins(
            GPIO_LEDS_USER_LED_1_PORT, GPIO_LEDS_USER_LED_1_PIN);
        DL_GPIO_clearInterruptStatus(GPIOA, GPIO_SWITCHES_USER_SWITCH_1_PIN);
    }
}
```

判断完 中断发生后 要及时清除中断标志位

volatile int32_t 全局变量监控 技巧

gpio 中断 注意设置的边沿

```
void GROUP1_IRQHandler(void)
{
    uint32_t gpioA = DL_GPIO_getEnabledInterruptStatus(GPIOA, GPIO_Encoder_PIN_Front_Right_A_PIN | GPIO_Encoder_PIN_Back_Right_A_PIN);
    uint32_t gpioB = DL_GPIO_getEnabledInterruptStatus(GPIOB, GPIO_Encoder_PIN_Front_Left_A_PIN | GPIO_Encoder_PIN_Back_Left_A_PIN);

    if (gpioA & GPIO_Encoder_PIN_Front_Right_A_PIN)
    {
        if(DL_GPIO_readPins(GPIO_Encoder_PIN_Front_Right_B_PORT, GPIO_Encoder_PIN_Front_Right_B_PIN))
            Front_Right_Count--;
        else
            Front_Right_Count++;
        DL_GPIO_clearInterruptStatus(GPIOA, GPIO_Encoder_PIN_Front_Right_A_PIN);
    }
}
```

系统时钟的配置

把主频提升到80MHz

注意勾选时钟树

然后图形化配置

printf重定向

选择Application的串口

§ DL_UART_transmitDataBlocking()

```
void DL_UART_transmitDataBlocking ( UART_Regs * uart,
                                   uint8_t    data
                                   )
```

Blocks to ensure transmit is ready before sending data.

Puts the data into the TX FIFO after blocking to ensure the TX FIFO is not full. Will wait indefinitely until there is space in the TX FIFO. See related APIs for additional transmit options.

Can be used for any data transfers that are less than or equal to 8 bits.

Parameters

[in] **uart** pointer to the register overlay for the peripheral

[in] **data** data to send

See also

[DL_UART_transmitData](#)

[DL_UART_transmitDataCheck](#)

Referenced by [DL_UART_getAnalogPulseWidth\(\)](#).

所传送的是ASCII码

为什么会显示在上位机上面 因为UART引脚通过跳线帽与板载调试器和上位机通信！

工程项目管理

从路径复制工程源码到workspace下面

所有的修改基于workspace下面

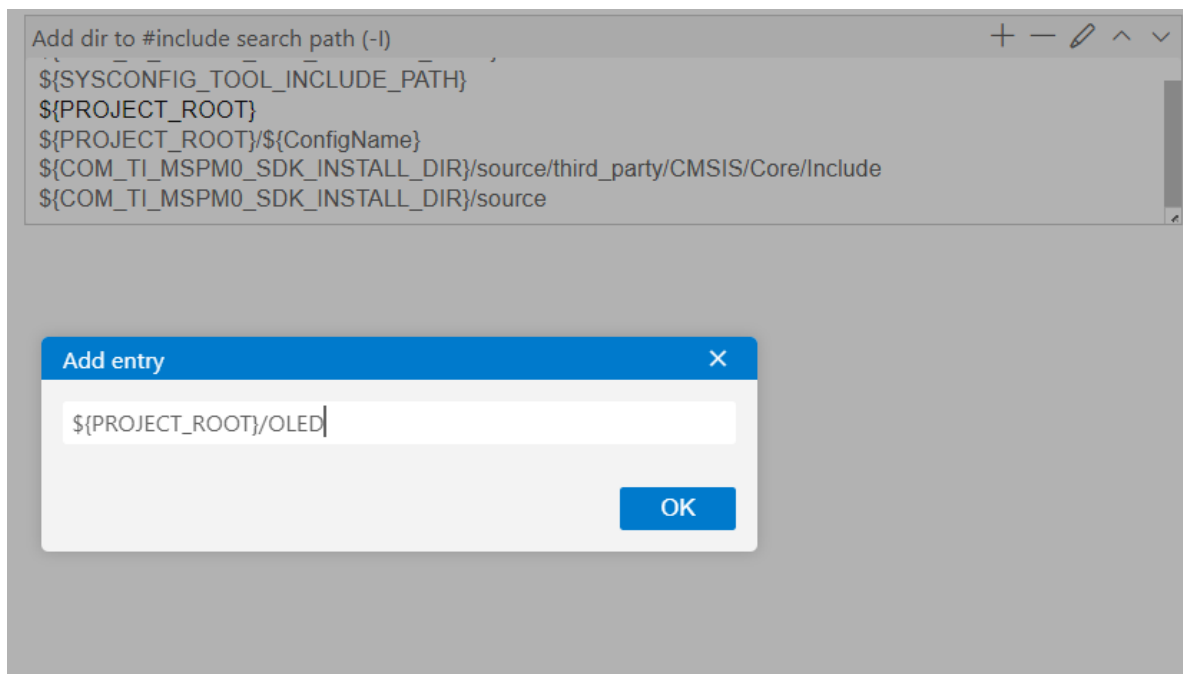
故源路径无法保存

要对workspace里面的文件夹进行复制和粘贴

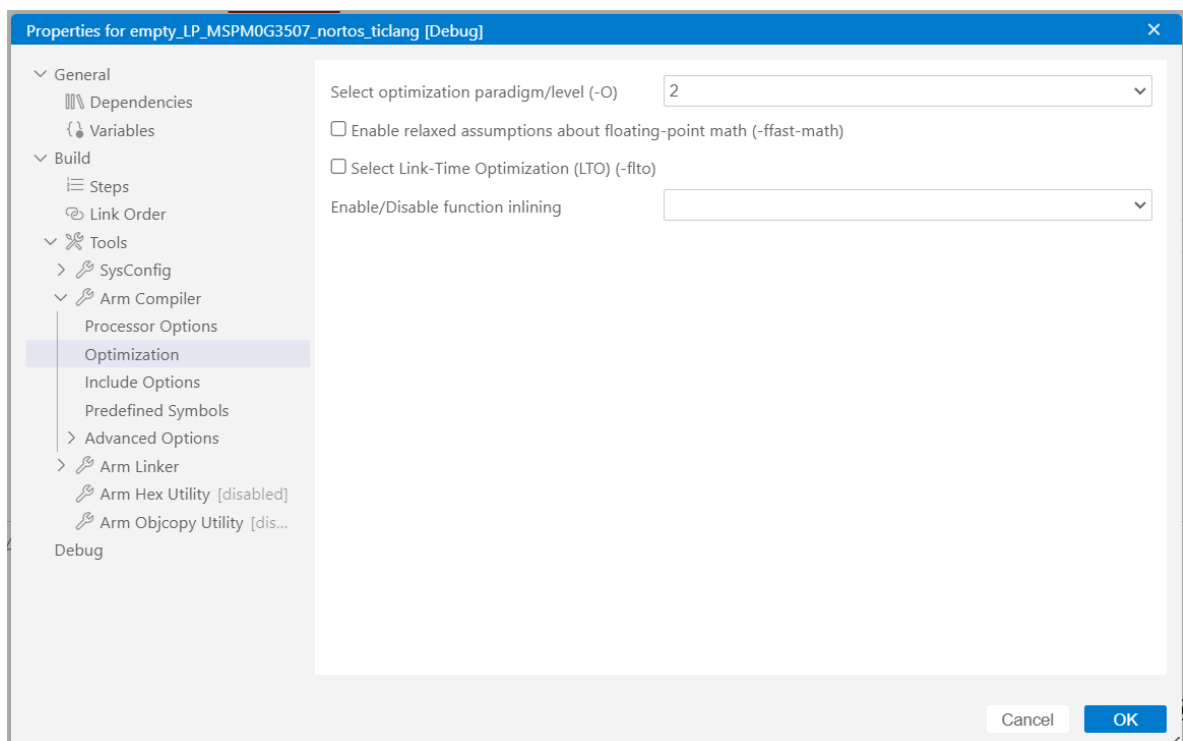
添加文件

添加文件夹

添加路径

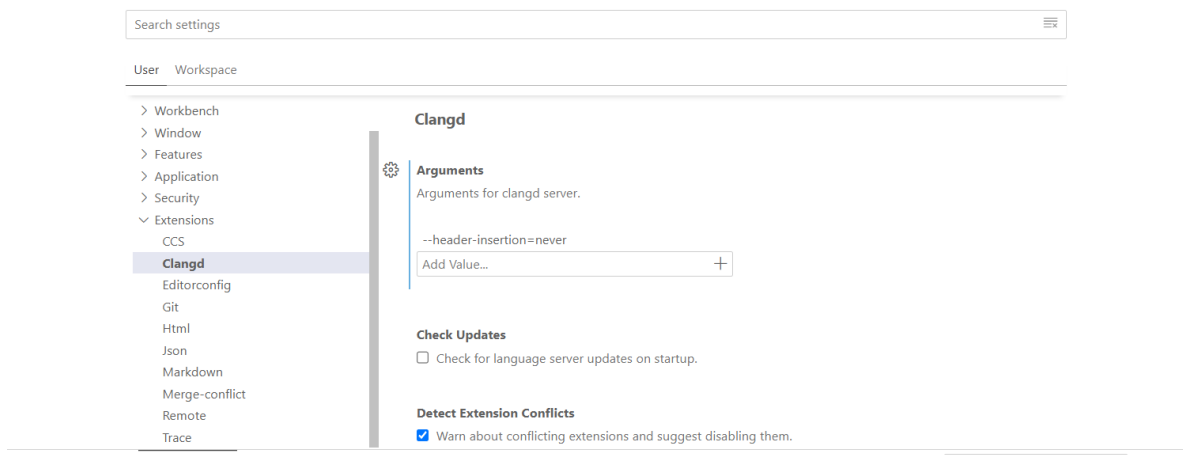


编译优化的说明

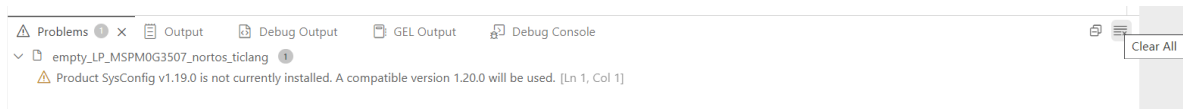


设置为 O (0)

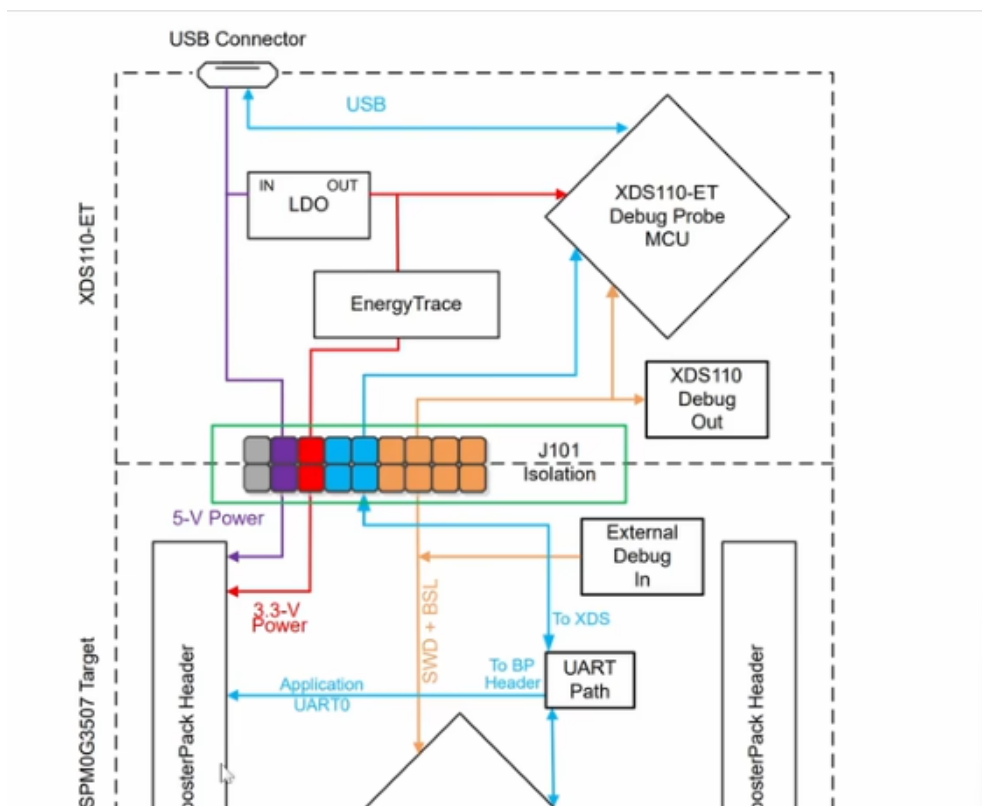
禁止自动添加没用的头文件



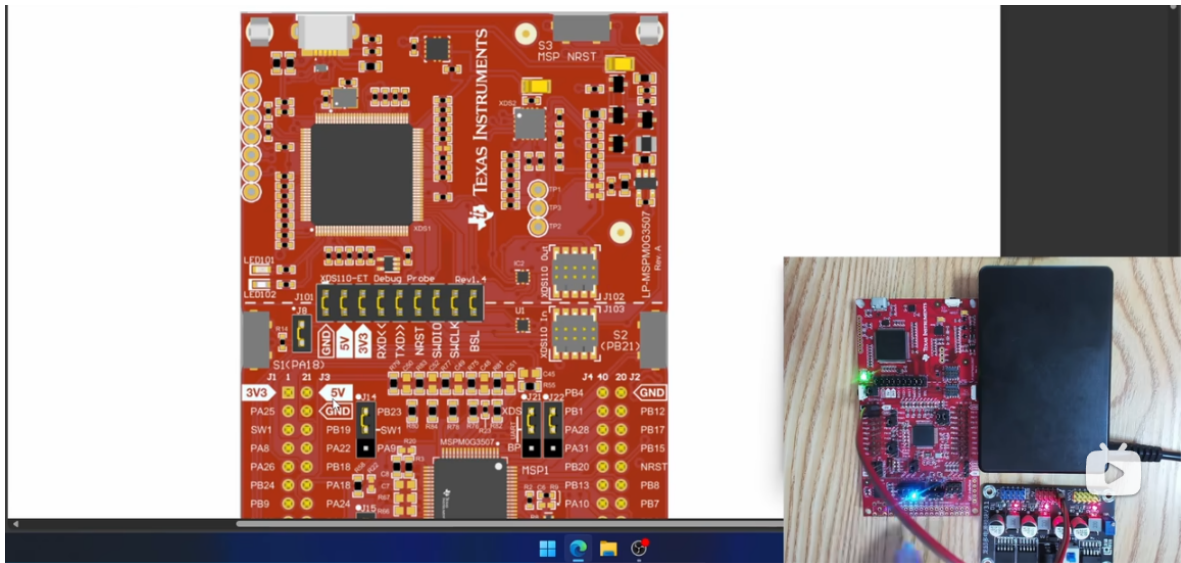
删除报错信息



LaunchPad外部供电



外部输入5V电压 (3.3V 不行)



引脚分配

J19 对于 PA0

J20 对于 PA1

PA0 和 PA1 是开漏输出

J19 和 J20 可以进行上拉

开漏输出上拉后可以输出高电平

J4 将LED1和PA0进行短接 如果PA0作为其他用途 则将跳线帽移除掉

PA2-PA6不能使用

PA9 和 PB23 只能用一个 SW1

PA10 和 PA11 通过跳线选择串口连到电脑上或者板子上面正常使用

PA16和PA18 只能用一个 SW2

PA18 轻易不要使用

按键主要使用 button S2

PA19和PA20 不能使用

PA21 相当于地

PA22 J16 连接光线传感器 如果使用 将跳线帽拔掉

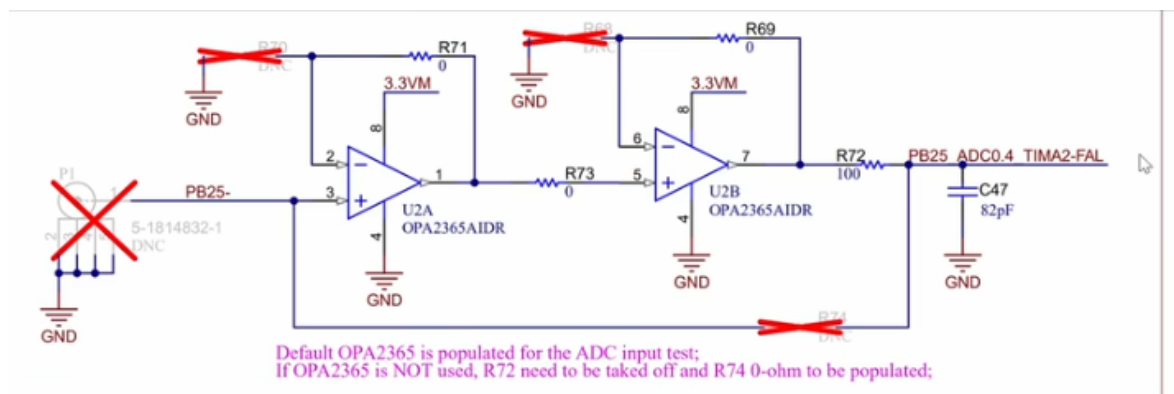
PA26 PA27同理

PA29 PA30 PB14 QEI

PB21 S2按键 要做一个上拉才能读取按键

PB22 跳线 RGD (LED2)

PB25 SMA



先分配 再连接

先特殊 后一般

OLED 软件I2C

Guangzhi Sun	
https://gitee.com/	guangzhi-sun ?
13942658971	
329012	37 秒后可重发
..... 👁	

我已阅读并同意 使用条款 及 非注册账号处理规则

移植模块：

添加新的文件夹

添加文件夹路径

include 不能忘记

OLED_init();

sysconfig也进行配置

(引脚名称 和 组的名称保持一致)

编译优化取消 (设置成0 0)

ADC进阶

Sample Clock Configuration

ADC Clock Source

SYSOSC

ADC Clock Frequency

32.00 MHz

Force SYSOSC Base Freq In STOP...

☐

Force SYSOSC Base Freq In RUN

☐

Sample Clock Divider

Divide by 1

Calculated Sample Clock Frequen...

32.00 MHz

改变这个无法改变采样率

Sampling Mode Configuration

Conversion Mode

Sequence

Conversion Starting Address

0

Conversion End Address (Sequence)...

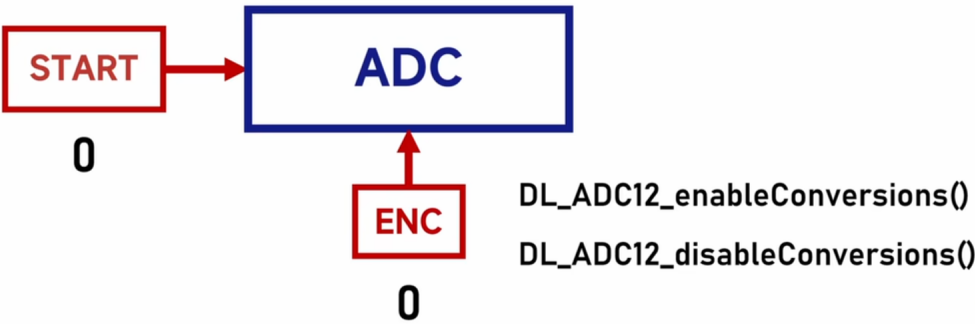
3

Enable Repeat Mode

☒

DL_ADC12_startConversion()

DL_ADC12_stopConversion()



单通道单词转换

初始化前 enable就是1

使能start

转换后

先调用enable 再调用start

单通道连续转换（勾选）

只用start一次就行

enable一直保持为1（这样转换完成后直接让start为1）

自己给自己触发信号

暂停单通道连续转换： 让enable=0

多通道单次转换

最后一个通道转换完成后 让enable=0

多通道多次转换

和单通道多次转换差不多

Sampling mode 选择 Auto

Power Down Mode ⓘ	Manual ▼
Desired Sample Time 0	0 ms
Actual Sample Time 0	0.00 s
Desired Sample Time 1	0 ms
Actual Sample Time 1	0.00 s

设想采集信号 采集1000个点 然后进入中断

DMA Configuration ^

Configure DMA Trigger	☒
Enable DMA Trigger	☒
DMA Samples Count	1
Enable DMA Triggers	None ▼

测一次传一个 所以count是1

Interrupt Configuration		^
Enable Interrupts	None	▼

DMA Configuration		^
Configure DMA Trigger	☒	
Enable DMA Trigger	☒	
DMA Samples Count	1	
Enable DMA Triggers	MEM0 result loaded interr...	▼

中断触发DMA

ADC寄存器地址不变 而数组下标地址不断改变

所以是Fixed to block

Transfer Mode ?	Single	▼
-----------------	--------	---

一对一的触发

Repeat 回到开头

Source Address Increment	Do not change address after...	▼
Destination Address Increment	Do not change address after...	▼

起点终点的地址不能increase

```

#include "ti/driverlib/dl_adc12.h"
#include "ti_msp_dl_config.h"

uint16_t gADCSamples[1000];

int main(void)
{
    SYSCFG_DL_init();
    DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) 0x40556280);
    DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &gADCSamples[0]);
    DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);

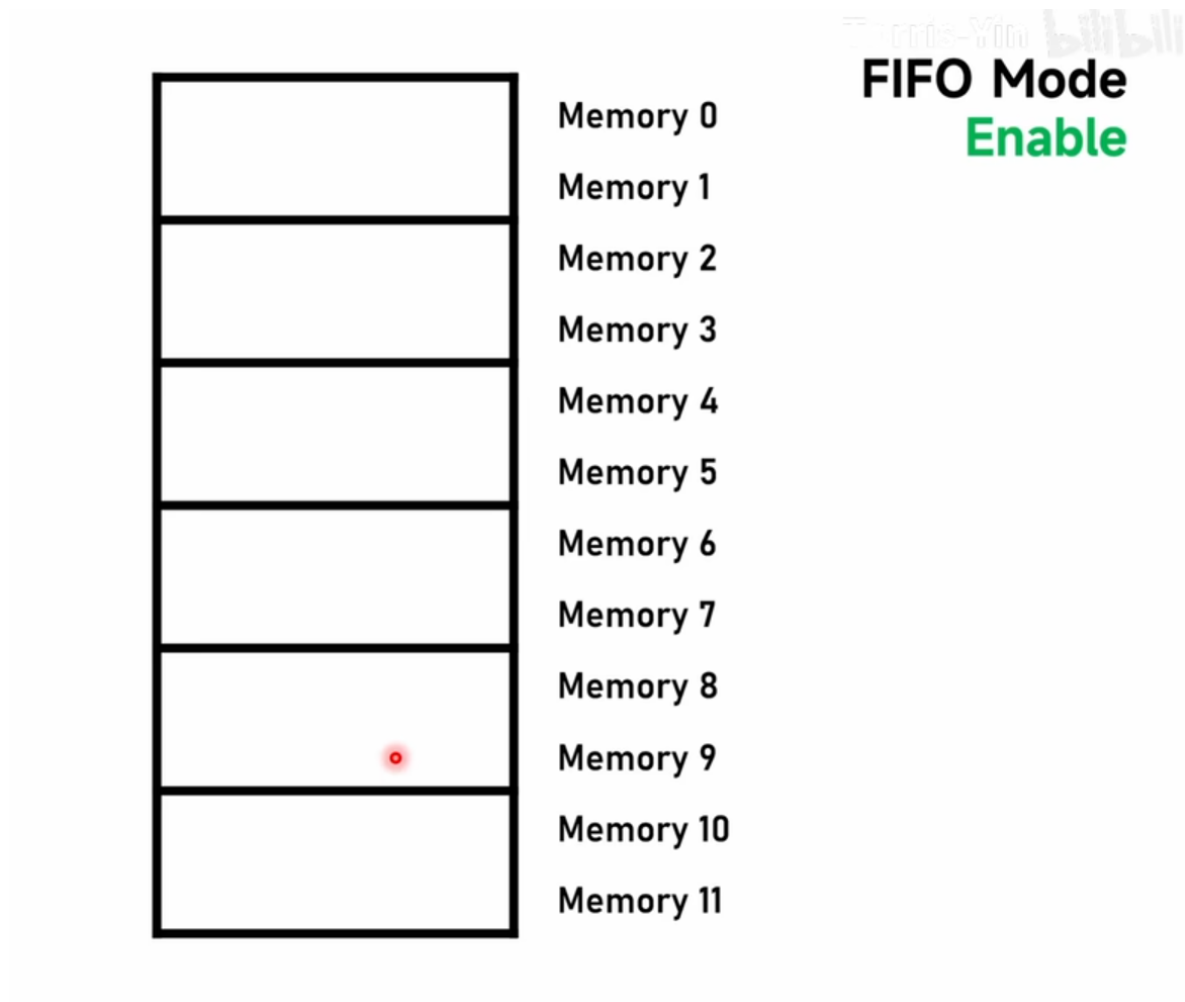
    NVIC_EnableIRQ(ADC12_0_INST_INT_IRQN);
    DL_ADC12_startConversion(ADC12_0_INST);

    while (1) {
    }
}

void ADC12_0_INST_IRQHandler(void)
{
    switch (DL_ADC12_getPendingInterrupt(ADC12_0_INST)) {
        case DL_ADC12_IIDX_DMA_DONE:
            DL_ADC12_disableConversions(ADC12_0_INST);
            //Process
            __BKPT();
            DL_ADC12_enableConversions(ADC12_0_INST);
            DL_ADC12_startConversion(ADC12_0_INST);
            break;
        default:
            break;
    }
}

```

FIFO 先进先出



16位 变成32位

fifo; 降低DMA和中断频率

多通道强制采集的时候也一定使用FIFO

定时器触发ADC

实现1MHz采样率

定时器1us

Basic Configuration

Clock Configuration

Timer Clock Source

BUSCLK

Timer Clock Divider

Divided by 1

Calculated Timer Clock Source

32000000

Timer Clock Prescaler

1

Calculated Timer Clock Values

Timer Clock Frequency

32.00 MHz

Timer Period Range And Resolutio...

31.25 ns to 2.05 ms w/ resolution of 4

Timer Mode

Periodic Down Counting

Desired Timer Period ⓘ

1us

Actual Timer Period

1.00 μs

Start Timer

☒

Advanced Configuration

Interrupts Configuration

Event Configuration

Event 1 Publisher Channel ID

1- TIMER_0 => ADC12_0

Event 1 Enable Controller Interrupts

Zero event

Event 2 Publisher Channel ID

Disabled (0)

Subscriber Port Selection

Disabled

Zero Counter On A Published Event

☐

Trigger Source

Event

Event Configuration

Event Publisher Channel ID

Disabled (0)

Event Subscriber Channel ID

1- TIMER_0 => ADC12_d

Enable Events

None

Trigger Mode

Valid trigger will step to nex...

ADC不受自己控制了

```

#include "ti_msp_dl_config.h"

uint16_t gADCSamples[1000];

int main(void)
{
    SYSCFG_DL_init();
    DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) 0x40556280);
    DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &gADCSamples[0]);
    DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);

    NVIC_EnableIRQ(ADC12_0_INST_INT_IRQN);

    while (1) {
    }
}

void ADC12_0_INST_IRQHandler(void)
{
    switch (DL_ADC12_getPendingInterrupt(ADC12_0_INST)) {
        case DL_ADC12_IIDX_DMA_DONE:
            DL_TimerA_stopCounter(TIMER_0_INST);
            //Process
            __BKPT();
            DL_TimerA_startCounter(TIMER_0_INST);
            break;
        default:
            break;
    }
}

```

如何配置最高的4M采样率

软件触发 DMA启动

Desired Sample Time 0	62.5 ns
Actual Sample Time 0	62.50 ns
Desired Sample Time 1	0 ms
Actual Sample Time 1	0.00 s

M0电源模式

RUN	SLEEP	STOP	STANDBY	SHUTDOWN
可用	CPU 和所有外设	所有外设	PD0 外设	除了 ADC、12 位 DAC 和 OPA 外，PD0 外设可用，最高 ULPClk 频率为 32kHz。
禁用	-	CPU 被禁用	CPU、SRAM 和 PD1 外设被禁用并保留（如果适用）	CPU、SRAM 和 PD1 外设被禁用并保留。高速振荡器（SYSPLL、HFXT、HFCLK_IN）和 SYSOSC 被禁用。

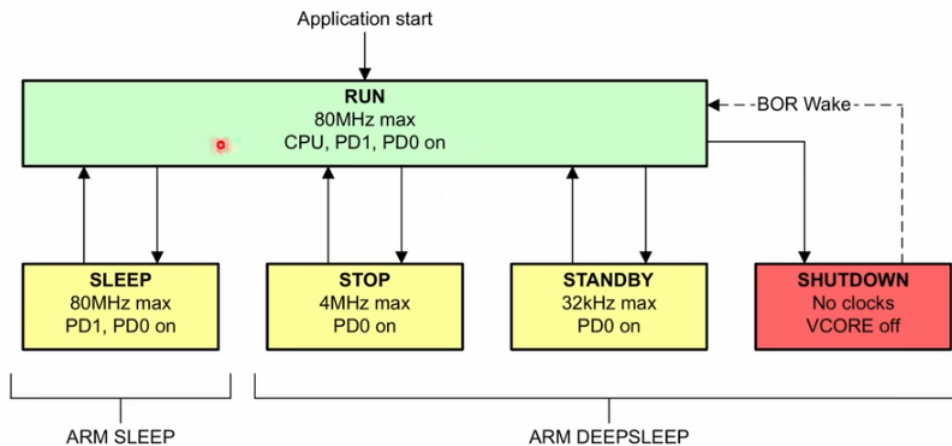


Figure 2-2. MSPM0Gxx Operating Modes

任务 1 - 在 SLEEP0 模式和 STANDBY0 模式之间交替

1.1 示例工程概述

该第一个代码示例展示器件在 SLEEP0 模式和 STANDBY0 模式之间的交替，以及配置相关外设的步骤：

- 启用用于驱动 LED 的 GPIO。
- 配置 TIMG 模块并设置计数模式（反复地向下计数）和周期。
- 启用 TIMG 中断。

在本实验室中，您将为 LED1（红灯）闪烁使用 GPIO 引脚 PA0，为 LED2（蓝灯）闪烁使用 GPIO 引脚 PB22。此外，您需要使用两个计时器（TIMG0 和 TIMG1）来设置 LED 闪烁频率。在不同的电源模式下，您会看到闪烁频率发生变化。TIMG1 时钟源自 SYSOSC，在 SLEEP 模式下有效，但在 STANDBY 模式下无效。TIMG0 时钟源自低频 LFOSC，因此在 SLEEP0 和 STANDBY0 模式下均有效。电源策略每 5s 交替一次。TIMG1 将在 SLEEP0 模式下每 0.25s 切换一次 LED1，并且在器件处于 STANDBY0 模式时不会切换。在 SLEEP0 和 STANDBY0 模式下，TIMG0 将以 1s 的间隔不断切换 LED2。

时钟频率=频率/（Divider*Prescaler）

Note

API `DL_SYSCTL_setPowerPolicySTANDBY0()` 和 `DL_SYSCTL_setPowerPolicyRUNOSLEEP0()` 实际上不会使器件进入 STANDBY0 或 SLEEP0 模式。在使用 API 设置电源策略之后，若要进入相应的模式，必须调用 `__WFE()` 来等待 ARM 事件，或调用 `__WFI()` 来等待中断。

`SYS_CFG_DL_SYSCTL_init()` 配置系统时钟，这决定了外设时钟源的频率。如果您想了解有关系统时钟的更多信息，请参阅数据表中的第 8.4 节“时钟模块 (CKM)”和技术参考手册中的第 2.3 节“时钟模块 (CKM)”。内部高频振荡器 SYSOSC 频率设置为 32MHz。ULPClk 源自 SYSOSC，分频比为 1，因此根据 `SYS_CFG_DL_SYSCTL_init()`，其频率设置为 32MHz。LFCLK 是低频时钟，其频率为 32768Hz。

对于 TIMG1，时钟源配置为 BUSCLK。TIMG1 位于 PD0 中，因此其时钟实际上源自 ULPClk。ULPClk 源自 SYSOSC (32MHz)。因此，在 SLEEP0 模式下，ULPClk 为 32MHz，但在 STANDBY0 模式下，ULPClk 为 32768Hz。与 TIMG0 配置类似，TIMG1 将反复地从周期值 31249 开始向下计数，当它向下计数到零时，会触发上述的 TIMG1 中断，因此 TIMG1 中断在 SLEEP0 模式下每 0.25s $((31249+1)/125000 = 0.25s)$ 被触发一次，在 STANDBY0 模式下每 254s $((31249+1)/123.2 = 254s)$ 被触发一次。

原来这里的TimerG1为sysconfig中的TimerG6

```

int main(void)
{
    SYSCFG_DL_init();

    gTogglePolicy = false;

    NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);
    NVIC_EnableIRQ(TIMER_1_INST_INT_IRQN);

    DL_TimerG_startCounter(TIMER_0_INST);
    DL_TimerG_startCounter(TIMER_1_INST);

    while (1) {
        while (false == gTogglePolicy) {
            __WFE();//WFE = wait for event 等待事件，即下一次事件发生前都在此hold住不干活
        }

        gTogglePolicy = false;

        DL_SYSCTL_setPowerPolicySTANDBY0();
        printf("standby\r\n"); //确实是进入了standby模式
        while (false == gTogglePolicy) {
            __WFE();
        }
        gTogglePolicy = false;

        DL_SYSCTL_setPowerPolicyRUN0SLEEP0();
        printf("sleep\r\n");
    }
}

/**
 * SLEEP0 Clock, should toggle faster in RUN0SLEEP0, toggles the LED slower
 * when running in STANDBY0
 */
void TIMER_1_INST_IRQHandler(void) //0.25秒触发一次
{
    switch (DL_TimerG_getPendingInterrupt(TIMER_1_INST)) {
        case DL_TIMERG_IIDX_ZERO:
            DL_GPIO_togglePins(GPIO_LEDS1_PORT, GPIO_LEDS1_USER_LED_1_PIN);
            printf("timer1\r\n");
            break;
        default:
            break;
    }
}

```

```

/**
 * STANDBY0 Clock, runs all the time at the same frequency
 */
void TIMER_0_INST_IRQHandler(void)           //1s被触发一次
{
    static uint32_t count = 5;
    switch (DL_TimerG_getPendingInterrupt(TIMER_0_INST)) {
        case DL_TIMERG_IIDX_ZERO:
            DL_GPIO_togglePins(GPIO_LEDS2_PORT, GPIO_LEDS2_USER_LED_2_PIN);
            if (count == 1) {
                gTogglePolicy = true;
                count          = 5;
            } else {
                count--;
            }
            break;
        default:
            break;
    }
}

```

忆往昔 工作的时候正常 不工作的时候进入sleep模式

```

int main(void)
{
    SYSCFG_DL_init();

    NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);
    DL_SYSCTL_enableSleepOnExit();

    DL_TimerG_startCounter(TIMER_0_INST);

    while (1) {
        __WFI();
    }
}

```

任务 3 – 从 SHUTDOWN 模式唤醒

3.1 示例工程概述

在本实验室中，您可以根据两个输入引脚（**PB21** 和 **PB12**）的状态来设置不同的电源状态。这可以防止器件无限期地停留在 SHUTDOWN 模式，因为串行线调试 (SWD) 在器件处于 SHUTDOWN 模式时保持锁定状态。

在进入 SHUTDOWN 模式后，当唤醒引脚 (**PA18**) 检测到上升沿时，器件将退出该状态。然后它将切换 **PB26**，从而使 ****LED2（红色）**** 的闪烁次数与器件被唤醒的次数相同。（即，当器件第一次被唤醒时，LED2 会闪烁一次，在器件第二次被唤醒时发生第二次闪烁，依此类推）。

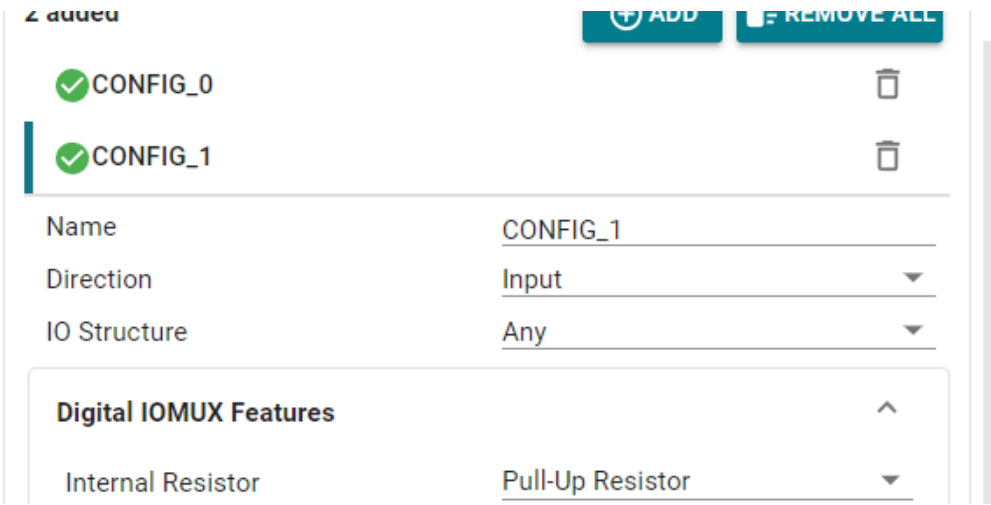
如果通过配置输入引脚选择了不同的电源策略，器件将保持所选的电源策略，但不会通过唤醒引脚唤醒，这意味着如果您未配置为 SHUTDOWN 模式，则将不会看到 LED2 闪烁。

基于引脚状态的电源策略配置如下：

配置引脚 0 (PB21)	配置引脚 1 (PB12)	电源策略
高电平（默认）	高电平（默认）	SHUTDOWN
高电平（默认）	低电平（引脚接地）	STOP0
低电平（引脚接地）	高电平（默认）	STANDBY0
低电平（引脚接地）	低电平（引脚接地）	RUN0SLEEP0

请按照以下步骤立即启动设计。

- 将 SYSCTL 示例工程 `sysctl_shutdown` 从 TI Resource Explorer 导入到 CCS 中：
 - MSPM0 SDK→examples→nonrtos→LP_MSPM0G3507→driverlib→`sysctl_shutdown`
 - 点击右上角的 **Import**
- 通过在 CCS 中启动调试会话来构建代码并对 LaunchPad 进行编程。



默认引脚上拉了

在进入 SHUTDOWN 模式后，当唤醒引脚 (**PA18**) 检测到上升沿时，器件将退出该状态

串行线调试 (SWD) 在器件处于 SHUTDOWN 模式时保持锁定状态。

- 将当前计数器值保存至 SYSCTL 中通用存储器的第一个字节，这样即使在 SHUTDOWN 模式下也可以跟踪器件被唤醒的次数。

Note

SYSCTL 寄存器提供了四个字节的通用存储器，即使在 SHUTDOWN 模式下也会被保留。这些存储器字节用于存储信息。

Wakeup Logic ⓘ

Wake when 1

Description

Wakeup Logic allowing a wakeup of the device when either:

- Pin changes to 0
- Pin changes to 1

Only available on Standard-Drive with wake, 5V tolerant Open Drain and High-Drive IO structures

Assigned Port	PORTA
Assigned Port Segment	Any
Assigned Pin ⓘ	18
Interrupts/Events	
Enable Interrupts	☐
Trigger Polarity	Trigger on Rising Edge
Event Publishing Channel	Disabled (0)

强上拉的引脚不能用set和clear赋值

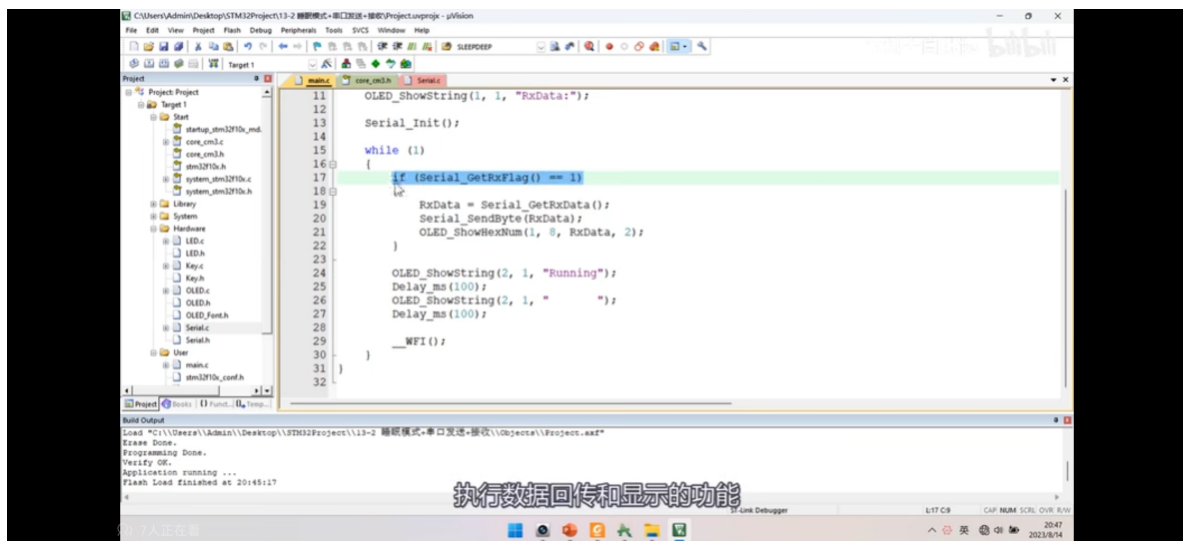
上电后自动复位一次 也就进入了shutdown模式

停止模式从中断的地方继续运行

待机模式从头开始运行

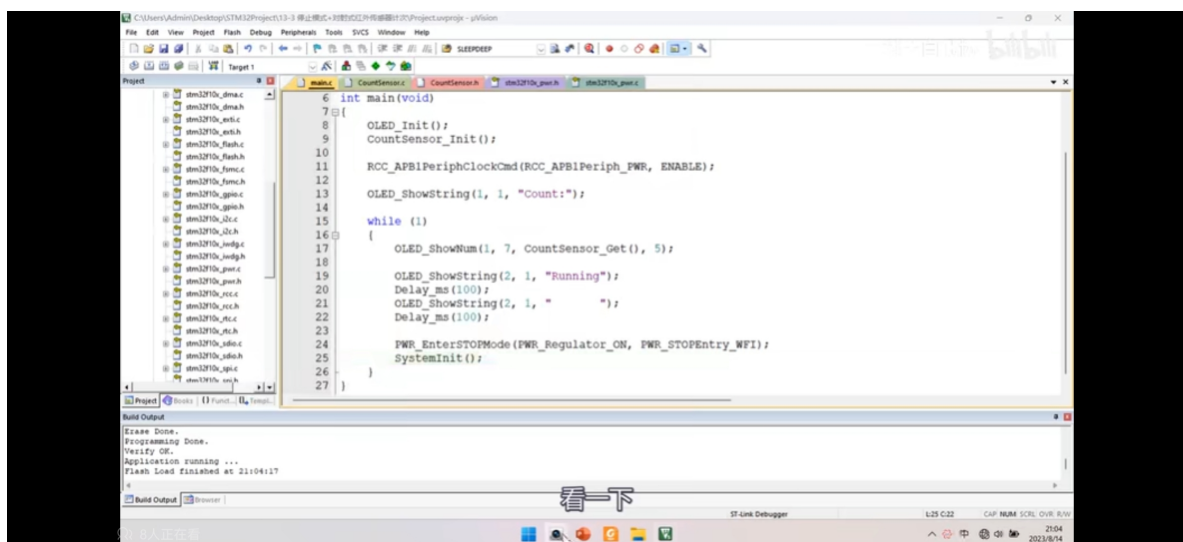
在待机模式下，所有IO引脚为高阻态（浮空输入）（对外不输出高低电平）

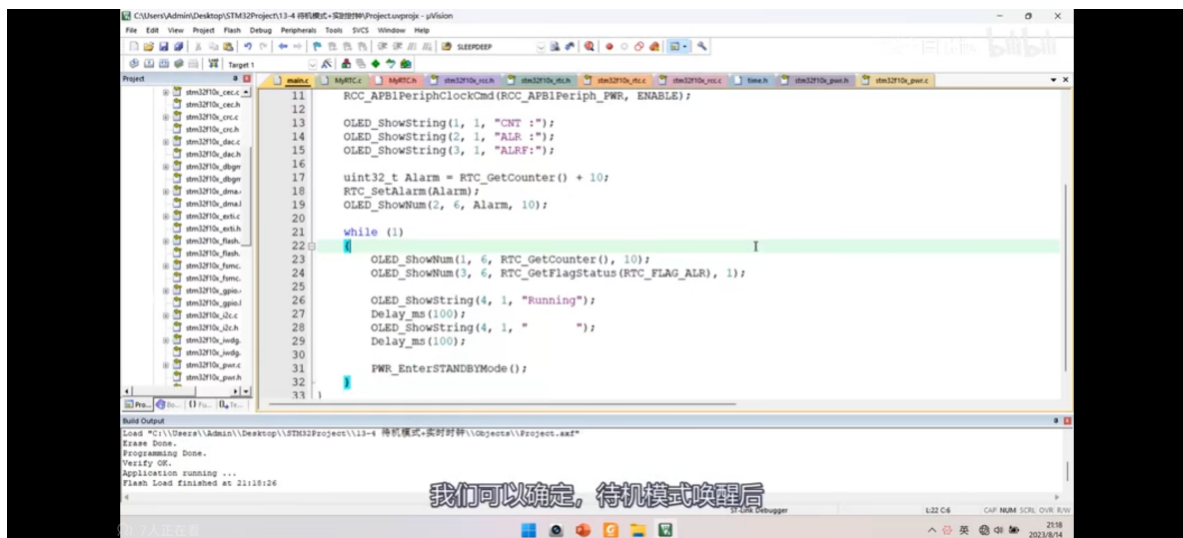
WAKE UP引脚上升沿退出待机模式



睡眠模式 在main函数最后放一个wfi指令

每来一个中断 先进入中断服务函数 再执行一遍程序的功能





待机模式结束后程序从头开始执行

```
SYSCFG_DL_init();

rstCause = DL_SYSCTL_getResetCause(); // 获取复位原因

if (DL_SYSCTL_RESET_CAUSE_BOR_WAKE_FROM_SHUTDOWN == rstCause)
/* Device was woken up from a SHUTDOWN state from this GPIO pin */
{
    /* Release IO after Shutdown before initializing any peripherals */
    SYSCFG_DL_GPIO_init();
    DL_SYSCTL_releaseShutdownIO();
    DL_GPIO_disableWakeUp(GPIO_SWITCH_USER_SWITCH_1_IOMUX);

    /* Load save state after wake from SHUTDNSTORE */
    counter = 1 + DL_SYSCTL_getShutdownStorageByte(DL_SYSCTL_SHUTDOWN_STORAGE_BYTE_0);
    //读取寄存器中的值

    /* Blink LED a number of times equal to counter */
    for (blink = 0; blink < (2 * counter); blink++) {
        DL_GPIO_togglePins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_2_PIN);
        delay_cycles(16000000);
    }
} else
```

```

} else
/* Device was NOT woken up from a SHUTDOWN state from this GPIO pin */
{
    counter = 0;
    DL_GPIO_setPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_1_PIN);
    delay_cycles(32000000);
    DL_GPIO_clearPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_1_PIN);
    printf("else");
}
printf("sgzcode\r\n");
/**
 * Resolve which Power policy to enable based on GPIO_INPUT_CONFIG_0_PIN and
 * GPIO_INPUT_CONFIG_1_PIN state.
 */
//DL_GPIO_togglePins(GPIO_INPUT_PORT, GPIO_INPUT_CONFIG_1_PIN);

switch (DL_GPIO_readPins(GPIO_INPUT_PORT,
    (GPIO_INPUT_CONFIG_0_PIN | GPIO_INPUT_CONFIG_1_PIN))) {
/**
 * GPIO_INPUT_CONFIG_0_PIN and GPIO_INPUT_CONFIG_1_PIN are not connected
 * (since inputs are internally pulled high the pins will be at Vcc).
 */
case (GPIO_INPUT_CONFIG_0_PIN | GPIO_INPUT_CONFIG_1_PIN):
/**
 * Configure Shutdown wake-up pin to wake-up when pin is set
 * to high before changing power policy to SHUTDOWN
 */
    DL_GPIO_initDigitalInputFeatures(GPIO_SWITCH_USER_SWITCH_1_IOMUX,
        DL_GPIO_INVERSION_DISABLE, DL_GPIO_RESISTOR_NONE,
        DL_GPIO_HYSTERESIS_DISABLE, DL_GPIO_WAKEUP_ON_1);
    DL_SYSCCTL_setPowerPolicySHUTDOWN();
    printf("shutdown\r\n");
    break;
/**
 * GPIO_INPUT_CONFIG_0_PIN is not connected and
 * GPIO_INPUT_CONFIG_1_PIN is connected to ground.
 */
case (GPIO_INPUT_CONFIG_0_PIN):
    DL_SYSCCTL_setPowerPolicySTOP0();
    printf("stop\r\n");
    break;

```

改变计数个数

```

static uint32_t count = TIMER_500_MILLISECONDS_TICKS; //495
switch (DL_TimerG_getPendingInterrupt(TIMER_0_INST)) {
    case DL_TIMER_IIDX_ZERO:
        /*
         * Counter stopped to avoid a conflict with the timer reading
         * the LOAD value while it's being set
         */
        DL_TimerG_stopCounter(TIMER_0_INST);
        /*
         * Count progressively gets smaller in 0.05 s increments until
         * reset with 0.5s
         */
        if (count > (TIMER_500_MILLISECONDS_TICKS / 5)) { //99
            count = count - TIMER_50_MILLISECONDS_TICKS; //50
        } else {
            count = TIMER_500_MILLISECONDS_TICKS;
        }

        DL_Timer_setLoadValue(TIMER_0_INST, count);
    /*

```

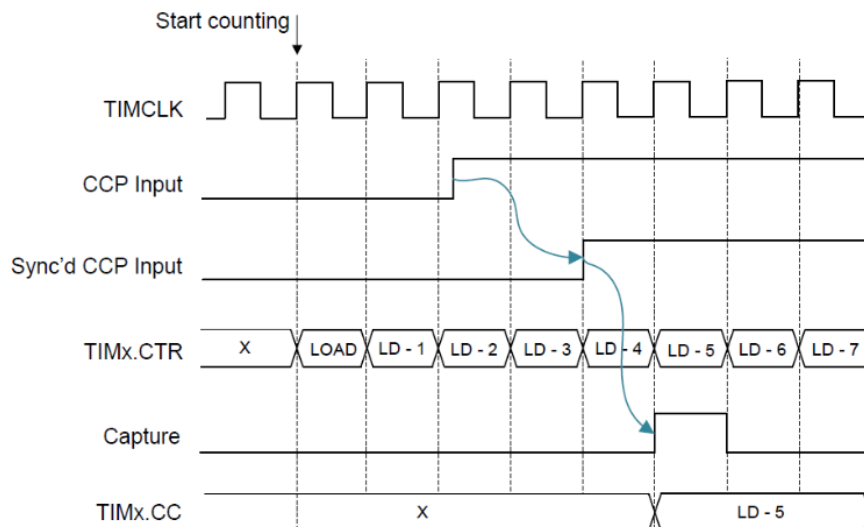
通用计时器实验室：捕获模式

将配置 MCU 并设置计时器以捕获外部输入信号并产生中断，从而获取有关外部输入信号的信息，例如脉冲数、频率和脉冲宽度。

边沿捕获：

边沿时间捕获功能测量从捕获操作开始到信号边沿的时间（以 TIMCLK 周期为单位）。计数器在 TIM 启用后加载，并递减每个 TIMCLK，直到检测到 CCP 边沿，这会触发对计时器值的捕获并生成捕获事件。然后，边沿时间等于 TIMx.LOAD 中的加载值减去 TIMx.CC_xy[0/1] 寄存器中的捕获值。

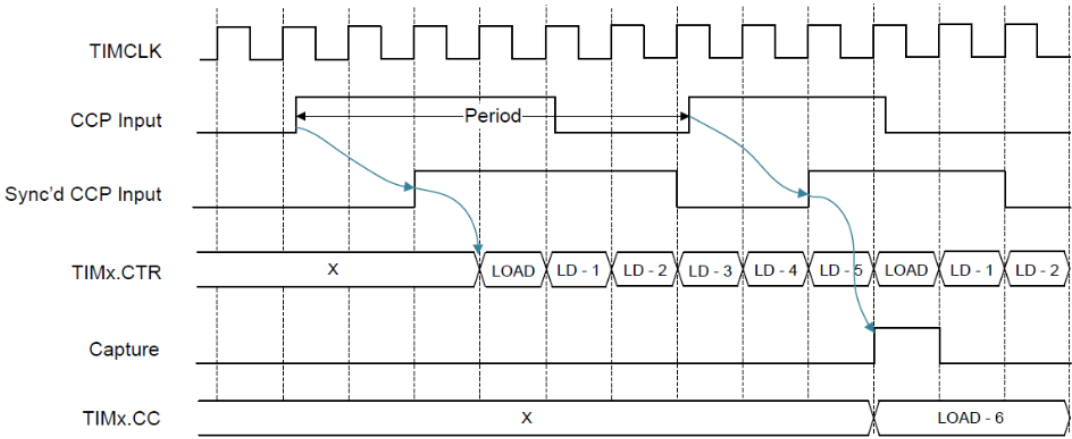
捕获边沿时间并将其放置在 CC_01[0] 寄存器中。



周期捕获：

周期捕获功能测量输入 CCP 上的信号周期。在 CCP 输入的每个正边沿上，将TIMx.CTR 值捕获到 TIMx.CC 寄存器中（生成捕获事件）并随 TIMx.LOAD 寄存器值一同加载。计数器在每个后续 TIMCLK 周期内递减。然后得到的信号周期为 TIMx.LOAD 中的加载值减去 TIMx.CC 中的捕获值。

捕获周期并将其放置在 CC_01[0] 寄存器中。



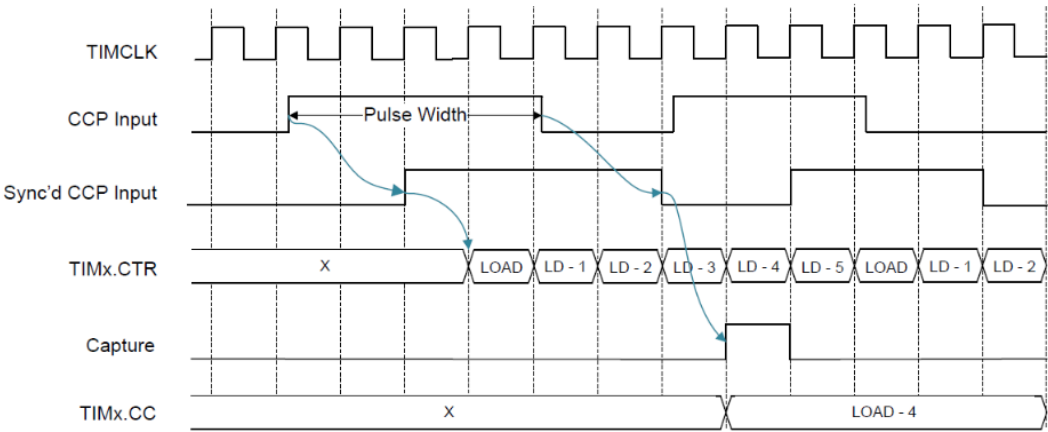
Error

注意：第一个周期边沿被丢弃。必须看到两个边沿后计时器才能够计算信号周期。

脉宽捕获：

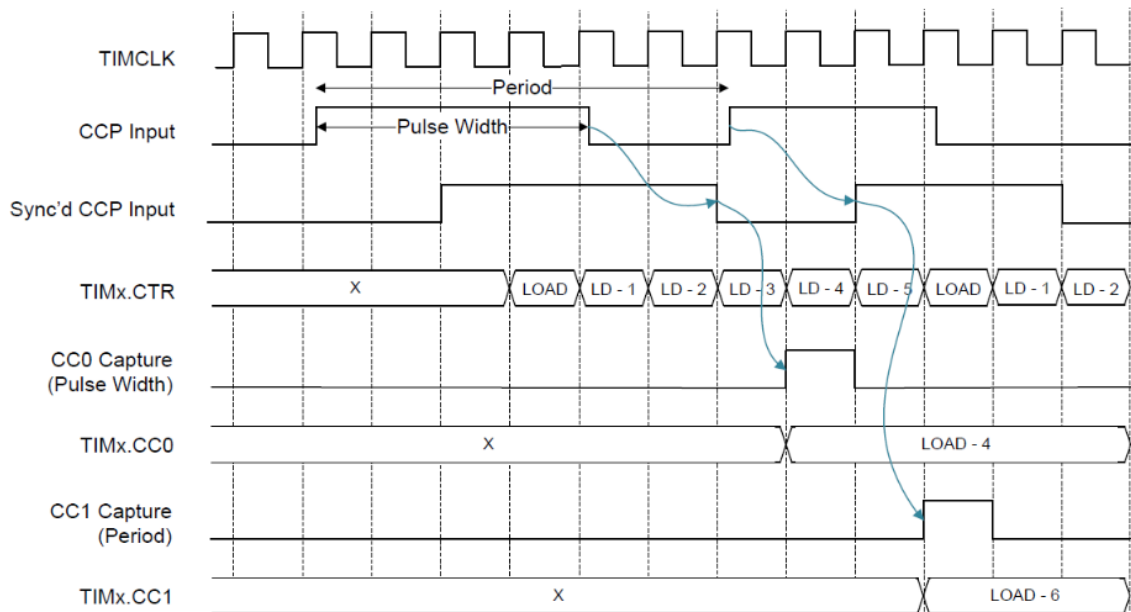
脉宽捕获功能测量 CCP 上信号的高电平时间。高电平时间是从上升沿到下降沿的 TIMCLK 周期数。计数器在正边沿被加载，并在负边沿被捕获（生成捕获事件）。零事件也可用于指示静态输入。

捕获脉冲宽度并将其放置在 CC_01[0] 寄存器中。



组合脉宽和周期捕获：

使用两个捕获寄存器可以同时捕获单个输入波形的脉宽和周期。寄存器 CC_0[0] 捕获脉冲宽度信息，而 CC_0[1] 捕获周期信息。



任务 1 – 导入、编程和编辑 TIMx 边沿捕获模式示例

任务 1 - 示例项目概述

本演示在 PA12 中捕获外部信号。TIMG0 配置为边沿捕获模式。本例中，计时器计数周期设为 2000。TIMCLK 使用设置为 32kHz 的 LFCLK。检测到外部信号边沿时，此代码将计算时间（从开始捕获到事件生成）。此时停止程序执行并使用调试器读出值。

在此任务中，没有要修改的代码。因此，使用代码示例编译 MSPM0G3507 LaunchPad 并对其进行编程。

检测电平跳变 执行动作

每次捕获后都应该把CNT清零

时钟频率就是驱动CNT的频率

任务 2 – 导入、编程和编辑 TIMx 占空比和周期捕获模式示例

任务 2 - 示例项目概述

本演示捕获外部信号（PA12）并使用 TIMG0 获取占空比和周期信息。在此示例中，TIMCLK 使用 BUSCLK 和预分频设置 255。捕获通道 0 (PA12) 用于捕获周期模式。检测到信号周期时，将计算周期和占空比。此时停止程序执行并使用调试器读出值。

在此任务中，没有要修改的代码。因此，使用代码示例生成 mspm0 LaunchPad 并进行编程。

理解清空再测试的方法对于CC_0

中断标志位 清除